TECHNISCHE UNIVERSITÄT DRESDEN

NETD

Faculty of Computer Science Institute of Systems Architecture

Chair of Distributed and Networked Systems

Complex Assignment for IoT Communications

Identifying Deployment Environments and IoT Sensor Nodes Using Link Quality Fluctuations

Duy Do Le (XXXXXXX)

Ngoc Thao Trang Ho (XXXXXXX)

Timon Althaus (XXXXXXX)

Yu-Ching Lai (XXXXXXX)

Yu Ling Zhong (XXXXXXX)

29th March 2026

Supervisor

Dr.-Ing. habil. Waltenegus Dargie

Contents

Symbols and Acronyms . . . III

1 Introduction . . . 1

2 System Deployment . . . 2
2.1 Hardware Components . . . 2
2.2 Network Topology . . . 2
2.3 Node Distances . . . 2

3 Implementation . . . 4
3.1 Communication Protocol . . . 4
3.1.1 BLE Configuration . . . 4
3.1.2 Packet Structure . . . 4
3.2 Data Collection . . . 5
3.2.1 Dataset Description . . . 5
3.2.2 Raw Data Characteristics . . . 5
3.2.3 Feature Description . . . 5
3.3 Machine Learning . . . 6
3.3.1 Model Description . . . 6
3.3.2 Training Configuration . . . 8
3.3.3 Data Preprocessing . . . 9
3.3.4 Varied Parameters . . . 9

4 Results and Discussion . . . 10

5 Conclusion . . . 14

Symbols and Acronyms

BLE	Bluetooth Low Energy
CNN	Convolutional Neural Network
ResNet	Residual Neural Network
RSSI	Received Signal Strength Indicator
IoT	Internet of Things

1 Introduction

The ability to uniquely identify IoT sensor nodes and their deployment environments is a challenging yet valuable problem in wireless sensing. Due to manufacturing imperfections, individual devices exhibit microscopic differences in their radio characteristics, which can be observed through link quality fluctuations such as RSSI (Received Signal Strength Indicator).

This project investigates whether these fluctuations can be used to classify (1) the deployment environment and (2) the individual sensor node. We deploy a BLE-based sensor network across five outdoor environments and apply two machine learning models (CNN and ResNet) to perform classification.

2 System Deployment

2.1 Hardware Components

The system consists of 5 Adafruit Feather devices running Bluetooth Low Energy (BLE): 4 transmitter nodes and 1 receiver node. Each node is battery-powered and can continuously transmit and receive BLE packets.

2.2 Network Topology

At any given time, one transmitter node broadcasts BLE packets while the receiver node listens and records RSSI values. This is repeated for each transmitter. The topology follows a star configuration, with the receiver at the center. We tried to arrange the nodes roughly at equal angles in a cross shape, but oftentimes the surroundings forced us to deviate from that shape quite a bit.

2.3 Node Distances

Before the main data collection, we conducted 10 experiments per environment to determine the maximum reliable BLE transmission distance. In each experiment, we varied the distance between nodes and recorded the furthest distance at which packets were consistently received at maximum transmission power. The raw measurements per environment are summarized below:

Table 2.1: Average maximum transmission distance per environment

Environment	Average Distance (m)
Bridge	3.23
Lake	9.32
Forest	23.94
River	10.39
Garden	26.79

3 Implementation

3.1 Communication Protocol

3.1.1 BLE Configuration

All nodes run on RIOT OS [Bac+18] and communicate using Bluetooth Low Energy (BLE). The 4 transmitter nodes (RIOT-BLE-0 to RIOT-BLE-3) broadcast packets simultaneously to the receiver at maximum transmission power. The receiver logs all incoming packets without sending acknowledgments (broadcast mode). The target transmission rate was set to 10 packets per second (interval = 100 ms). This was confirmed by the measured median inter-packet interval of 100 ms across all nodes. However, the effective received rate varied per node and environment due to packet loss from distance and environmental interference (table 3.1):

Table 3.1: Effective received packet rate (pkt/s) per node per environment

Environment	RIOT-BLE-0	RIOT-BLE-1	RIOT-BLE-2	RIOT-BLE-3
Bridge	9.83	9.41	4.82	9.52
Forest	8.32	5.78	9.80	7.17
Garden	8.86	9.95	7.90	7.15
Lake	9.40	4.54	4.15	5.72
River	9.87	9.29	9.58	8.86

3.1.2 Packet Structure

Table 3.2: BLE packet structure as recorded by the receiver

Field	Type	Description
`ts`	Timestamp (ms)	Time of reception at the receiver
`device`	String	Transmitter node ID (e.g., RIOT-BLE-0)
`seq`	Integer	Packet sequence number (used for loss estimation)
`rssi`	Integer (dBm)	Received Signal Strength Indicator

3.2 Data Collection

3.2.1 Dataset Description

Data was collected across 5 outdoor deployment environments: Bridge, Forest, Garden, Lake, and River. In each environment, all 4 transmitter nodes (RIOT-BLE-0, RIOT-BLE-1, RIOT-BLE-2, RIOT-BLE-3) broadcast BLE packets simultaneously to the receiver node. Each session lasted approximately 30 minutes.
We used various locations in Dresden. As a Bridge we used the "Campus Bridge" over the "Bergstraße", our lake was the "Carolasee" in Dresden's "Großer Garten", the forest location was in the outskirts of the "Dresdner Heide", the river measurement was recorded at the side of the river "Elbe" and we used the publicly accessible garden of "Schloss Albrechtsberg" for the Garden. We expect notable interference especially at the bridge, from cars passing underneath, and at the riverside, since in the late afternoon when we took measurements, there were lots of people with smartphones passing by.

Environment	Date	Duration (min)	Total Packets
Bridge	06 Mar 2026	40.3	79,729
Lake	06 Mar 2026	31.7	45,106
Forest	07 Mar 2026	41.4	77,089
River	07 Mar 2026	33.4	75,307
Garden	07 Mar 2026	31.2	63,391
		Total	340,622

3.2.2 Raw Data Characteristics

Before processing the data for machine learning, we analyzed its distribution. Figure 3.1 and 3.2 illustrate the percentage of collected packets across different environments and nodes, showing a balanced dataset.

Furthermore, we examined the RSSI range for each environment. As shown in Figure 3.3, the signal strength varies significantly depending on the surroundings, which provides a basis for environment classification.

3.2.3 Feature Description

The RSSI statistics per node per environment are summarized below:

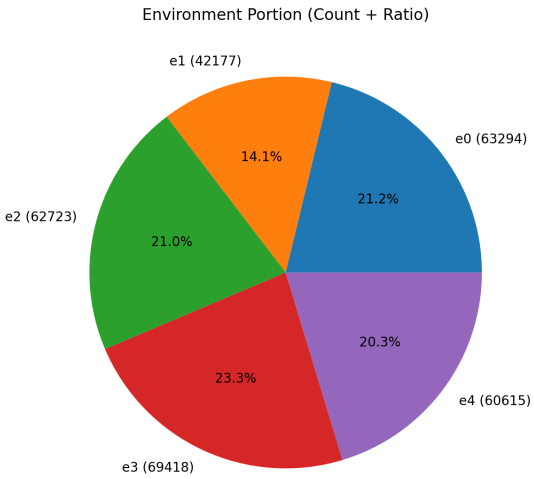

Figure 3.1: Portion of packets per environment

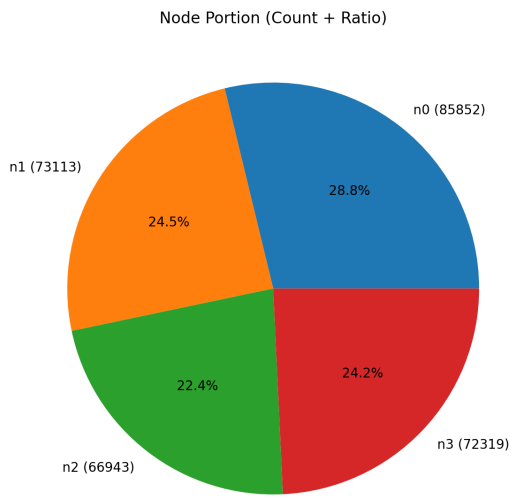

Figure 3.2: Portion of packets per node

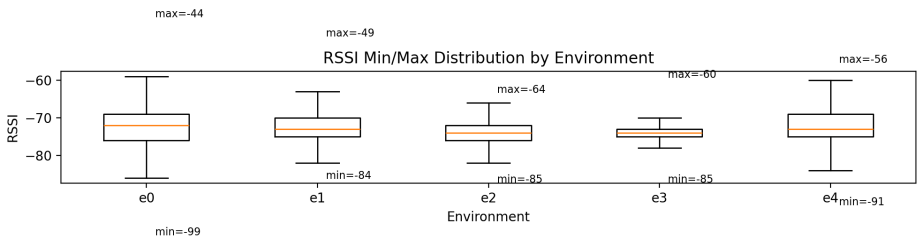

Figure 3.3: Minimum and Maximum RSSI values across environments

3.3 Machine Learning

3.3.1 Model Description

CNN1D Architecture

The CNN1D model consists of three convolutional blocks followed by a fully connected classifier. Each convolutional block includes a 1D convolution layer, batch normalization, ReLU activation, and max pooling.

The total number of trainable parameters in CNN1D is 52,676.

Table 3.3: RSSI statistics per node per environment

Environment	Node	Packets	Mean (dBm)	Min	Max	Std
Bridge	RIOT-BLE-0	23,747	-72.3	-98	-60	3.40
	RIOT-BLE-1	21,991	-74.5	-91	-44	3.60
	RIOT-BLE-2	11,641	-77.0	-99	-60	2.74
	RIOT-BLE-3	22,350	-66.7	-100	-54	4.53
Forest	RIOT-BLE-0	20,646	-74.3	-89	-62	3.40
	RIOT-BLE-1	14,343	-75.3	-90	-66	3.56
	RIOT-BLE-2	24,321	-73.7	-89	-64	3.32
	RIOT-BLE-3	17,779	-74.9	-90	-60	3.73
Garden	RIOT-BLE-0	16,585	-74.5	-87	-65	2.37
	RIOT-BLE-1	18,629	-67.2	-82	-60	2.92
	RIOT-BLE-2	14,787	-73.3	-91	-56	3.48
	RIOT-BLE-3	13,390	-75.1	-86	-66	2.06
Lake	RIOT-BLE-0	17,836	-69.8	-82	-49	3.52
	RIOT-BLE-1	8,515	-74.6	-83	-65	2.08
	RIOT-BLE-2	7,899	-74.4	-83	-66	2.00
	RIOT-BLE-3	10,856	-73.9	-84	-63	2.42
River	RIOT-BLE-0	19,788	-74.3	-85	-67	1.20
	RIOT-BLE-1	18,631	-70.2	-87	-60	2.73
	RIOT-BLE-2	19,123	-75.5	-83	-70	1.58
	RIOT-BLE-3	17,765	-74.9	-85	-63	1.89

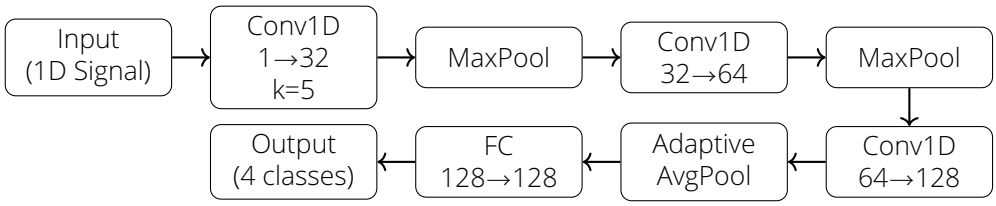

Figure 3.4: CNN1D architecture for sensor-based time-series classification.

ResNet1D Architecture

To enhance feature extraction capability and improve gradient flow, we extend the CNN architecture with residual connections [He+16], forming a ResNet1D model. Residual blocks allow the network to learn identity mappings and alleviate the vanishing gradient problem.

Each residual block consists of two convolutional layers with a skip connection:

$$\mathbf{y} = F(\mathbf{x}) + \mathbf{x} \tag{3.1}$$

where $F(\mathbf{x})$ represents the residual mapping.

The total number of trainable parameters in ResNet1D is 183,044

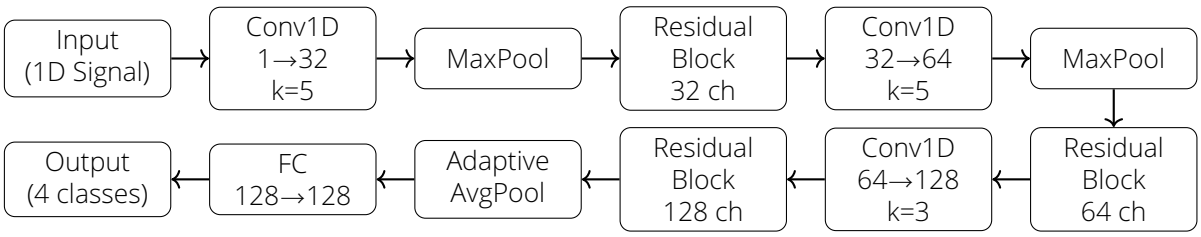

Figure 3.5: ResNet1D architecture for sensor-based time-series classification.

3.3.2 Training Configuration

The models were trained using the PyTorch framework on either GPU or CPU depending on hardware availability. The input time-series data were reshaped into a three-dimensional format of (batch, channel, length) to match the requirements of 1D convolutional layers.

Training Settings

Both CNN1D and ResNet1D models were trained using the following configuration:

- **Optimizer:** Adam
- **Learning rate:** 1×10^{-4}
- **Batch size:** 64
- **Loss function:** Cross-Entropy Loss
- **Number of epochs:** up to 100

Training Stabilization and Regularization

- To improve training stability, a learning rate scheduler (ReduceLROnPlateau) was applied, which reduces the learning rate by a factor of 0.5 if the validation loss does not improve for 3 consecutive epochs.
- To prevent overfitting, dropout with a rate of 0.3 was applied in the fully connected layers. In addition, an early stopping strategy was used based on validation loss with a patience of 20 epochs. The best model was selected according to the lowest validation loss and saved for final evaluation.

Evaluation Metrics

Model performance was evaluated using classification accuracy and macro-averaged F1-score, which is particularly suitable for handling potential class imbalance. A confusion matrix was also computed to provide detailed insights into class-wise prediction performance.

3.3.3 Data Preprocessing

The data for each environment and receiver node was processed in the following way: The received packets were ordered by timestamp and then the difference between two consecutively measured RSSI values was taken as a data point. These data points were then scaled to $[0, 1]$ using min-max normalization:

$$y' = \frac{y - y_{min}}{y_{max} - y_{min}} \tag{3.2}$$

Then those values were divided into overlapping windows according to the selected window size and overlap and each window was labeled by node or environment respectively.

3.3.4 Varied Parameters

Besides the model architecture (CNN/ResNet) we also varied whether we classified the node, which was receiving a given window, or the whether we classified the environment, in which a given window was received.
We also tested two different approaches to test validation. We either set a random quarter of all windows for testing apart, or we excluded all the data from one environment when classifying receiving nodes (when classifying environments, we set apart one node's entire data to get an equivalent setup).
Furthermore, we tried different window sizes and overlaps, thus setting the overlap on each side of the window to either 40% or 50% and setting the window length to either 100, 500 or 1000.
All these combinations lead to 48 different experimental setups, summarized in table 3.4.

Model	CNN	ResNet	
Task	classify node	Classify environment	
Test/Training split	random 25%	oneout	
Window size	100	500	1000
Overlap	40%	50%	

Table 3.4: The training combinations lead to 48 training scenarios, which were compared.

4 Results and Discussion

When classifying environments and holding back node1's data as test data, both model architectures converged to a state, which didn't produce good predictions. In this scenario the test accuracy always reached a final state between 40% and 45%, where it would always predict some environments correctly, but predict others almost always wrong. This is shown in the confusion matrices in figure 4.1. In the early epochs of training test loss would start to increase as training loss decreased, leading to the selection of a model that was trained for only a few epochs as the "best model".

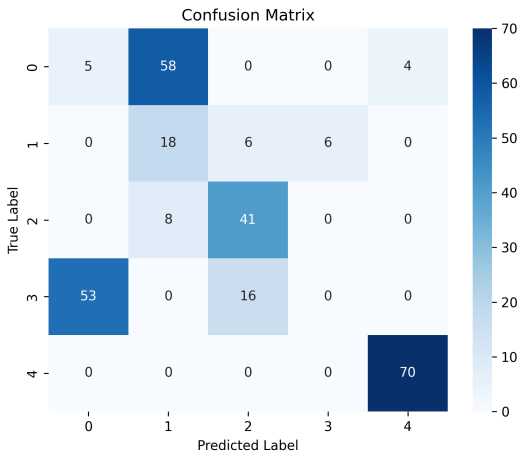

Figure 4.1: The confusion matrix after training the CNN for 140 epochs with one node's data excluded (window size 500, overlap 50%)

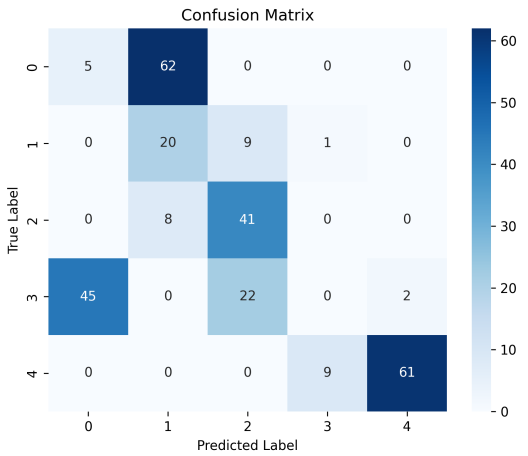

Figure 4.2: The confusion matrix after training the ResNet for 140 epochs with one node's data excluded (window size 500, overlap 50%)

A look at the confusion matrices of these selected models (an example is shown in figure 4.3), however, shows that environment 0 (bridge) and 3 (riverside) often get confused for each other. On the other hand, environments 1 (lake), 2 (forest) and 4 (garden) are easily confused with each other, but the forest is slightly less often misclassified. This seems rather plausible, as the bridge (0) and the riverside (3) featured the most interference from bypassing people, presumably carrying electronic devices. The forest (2) was the most secluded from possible interference, while in the garden (4) and at the lake (1), there were occasionally people passing by, but way less compared to the constant stream of traffic at the bridge or on the riverside.

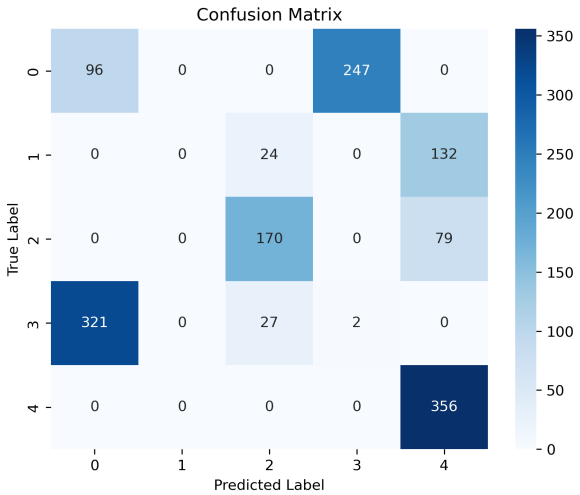

Figure 4.3: The best model at classifying the environment for a sequence length of 100 and an overlap of 50%, using a CNN, when using one node's data as test data. This training/test split of the data leads to rather bad guesses by the neural net, but the confusions are mostly between plausibly similar environments.

This observation seems to validate the idea that the neural net is indeed learning the patterns produced by electromagnetic interference in a given environment. But since the training set is still comparatively small, the loss landscape still has some unwanted features, which means that the model approaches a partially nonsensical state. All of these quirks disappeared when instead of excluding a node, a random quarter of all windows is used as the test data. Now the models converge to a state of ca. 75-90% test accuracy (figures 4.4, 4.5).

There are some minor differences in the other parameters, for example the ResNet is always slightly better at converging than the CNN as shown in figures 4.6, 4.7. The difference in window sizes and overlaps had no consistent impact on model performance as is summarized in figure 4.8.

In order to test against effects of data leakage, we tried splitting the data windows not randomly, but by time, using the last quarter as test data. Due to the overlaps, there likely are sequences of the test data less than the overlap size present in the training data, if random windows are excluded. This was done for a sequence length of 100 and an overlap of 50, where the effects would be most pronounced, for the example of a CNN classifying nodes. The result was a drop in test accuracy from 77% to 69%, that is a noticeable, but not fundamental change. However we also cannot rule out, that the actual environment changed on the timescale of minutes, so that the data was indeed different towards the end of the recording.

For the node classification we got similar results. Using a random 25% as the set apart test data led to good looking results, while excluding an environment from the training data made the model converge to nonsensical states where the test accuracy was barely different from random guessing. The full data, confusion matrices and training curves for every scenario can be found in the github repository.

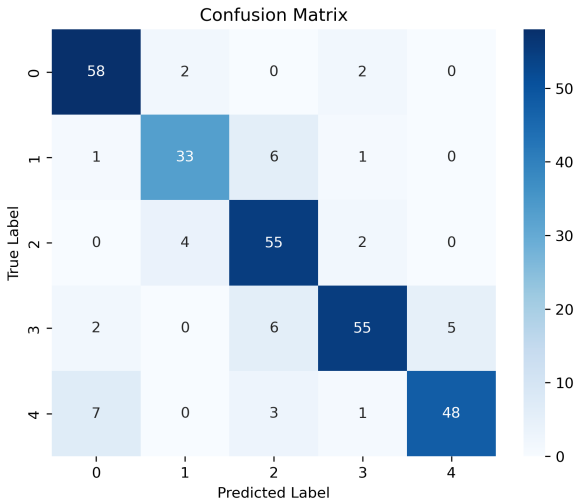

Figure 4.4: The CNN's ability at classifying an environment if trained on 500-wide windows with 50% overlap is good after having been trained on a random 75% of all data.

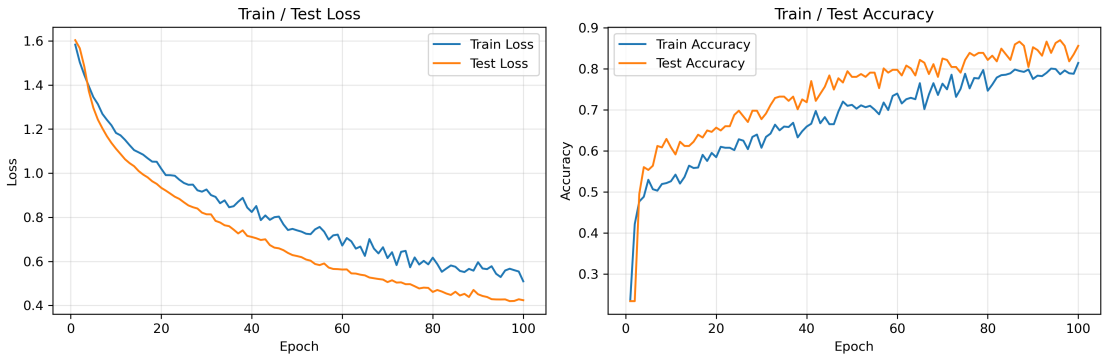

Figure 4.5: The corresponding training curves, show that test and training loss/accuracy behave similar.

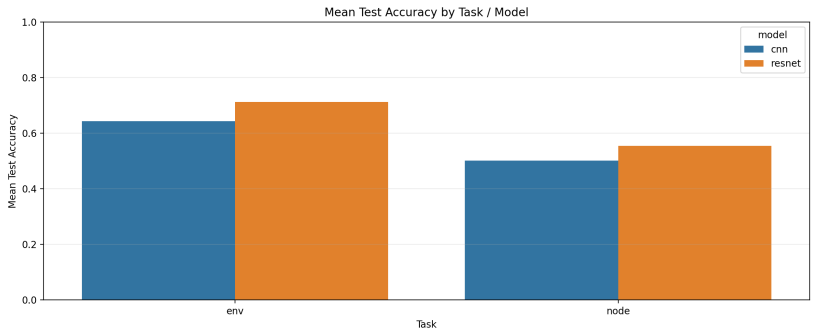

Figure 4.6: The mean test accuracy for the CNN and the ResNet compared.

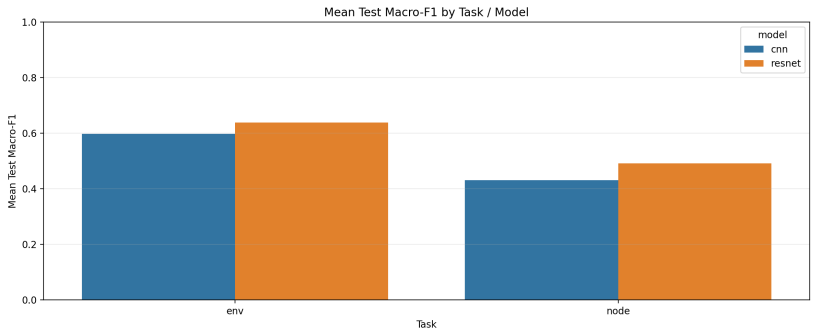

Figure 4.7: The mean F1-score compared between CNN and ResNet

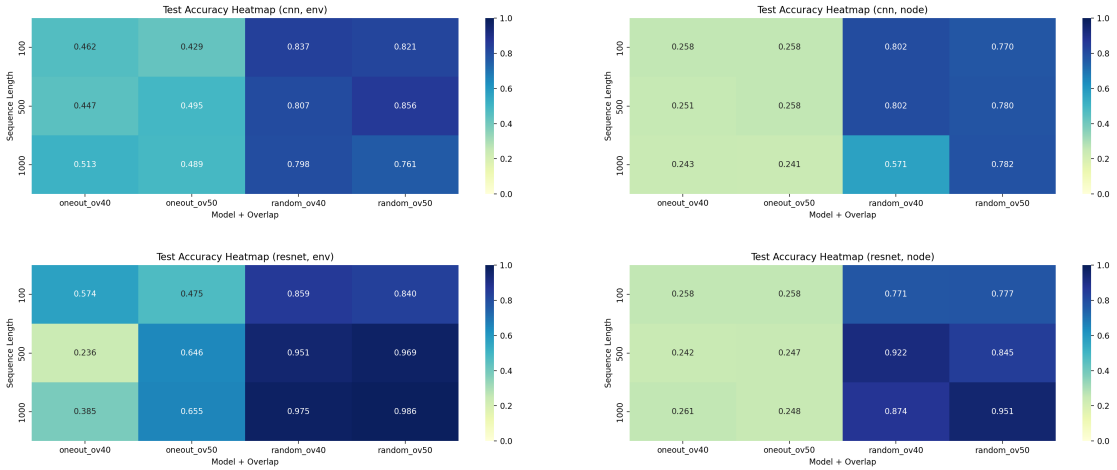

Figure 4.8: The Heatmaps showing the effects of different sequence length windows and overlaps; that is there is no large and consistent effect.

5 Conclusion

It seems possible to identify the node and the environment by use of a neural net trained on the RSSI time series we collected. However, excluding one full set of measurements from the training data showed that the models obtained this way were not very robust in new contexts. This is raising doubts whether the models trained this way would hold up in evaluating data from another run of measurements. But since even those results were not entirely unrealistic, there is reasonable hope that, on a bigger dataset with more runs in each environment, a model could be trained that can reliably identify environments or nodes, even in runs excluded from the training data.

Bibliography

[He+16] Kaiming He et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.

[Bac+18] Emmanuel Baccelli et al. "RIOT: An Open Source Operating System for Low-End Embedded Devices in the IoT". In: *IEEE Internet of Things Journal* 5.6 (2018), pp. 4428–4440. DOI: 10.1109/JIOT.2018.2815038.

List of Figures

3.1 Portion of packets per environment 6
3.2 Portion of packets per node 6
3.3 Minimum and Maximum RSSI values across environments 6
3.4 CNN1D architecture for sensor-based time-series classification. 7
3.5 ResNet1D architecture for sensor-based time-series classification. 8

4.1 The confusion matrix after training the CNN for 140 epochs with one node's data excluded (window size 500, overlap 50%) 10
4.2 The confusion matrix after training the ResNet for 140 epochs with one node's data excluded (window size 500, overlap 50%) 10
4.3 The best model at classifying the environment for a sequence length of 100 and an overlap of 50%, using a CNN, when using one node's data as test data. This training/test split of the data leads to rather bad guesses by the neural net, but the confusions are mostly between plausibly similar environments. 11
4.4 The CNN's ability at classifying an environment if trained on 500-wide windows with 50% overlap is good after having been trained on a random 75% of all data. 12
4.5 The corresponding training curves, show that test and training loss/accuracy behave similar. 12
4.6 The mean test accuracy for the CNN and the ResNet compared. 12
4.7 The mean F1-score compared between CNN and ResNet 12
4.8 The Heatmaps showing the effects of different sequence length windows and overlaps; that is there is no large and consistent effect. 13

List of Tables

2.1 Average maximum transmission distance per environment 3

3.1 Effective received packet rate (pkt/s) per node per environment 4
3.2 BLE packet structure as recorded by the receiver 4
3.3 RSSI statistics per node per environment 7
3.4 The training combinations lead to 48 training scenarios, which were compared. 9